159.102 Notes

Section 8 : Recursion, Searching and Backtracking

Recursion occurs when something is defined in terms of itself. The most common application of recursion is in mathematics or computer science. However, recursion is used in a range of disciplines including linguistics and language. For example, there is a rumour that the definition of recursion in a particular dictionary looks like this: Recursion: *see Recursion*.

A ***recursive function*** is a function that calls itself. Similarly a ***recursive definition*** is a definition that refers to itself. Sometimes we need to solve problems where a recursive solution is simpler and more natural than the alternative.

An example of a ***recursive definition*** is the definition of the set of natural numbers, which is:
1. if n is in the set of natural numbers then n + 1 is in the set of natural numbers
2. 0 is in the set of natural numbers

(You will be pleased to know that the definition of the set of natural numbers has nothing to do with our course!)

A recursive definition of factorial

Factorial is defined for zero and positive integers. The symbol for factorial is !

Examples of factorial are: 5! = 5 * 4 * 3 * 2 * 1 and 4! = 4 * 3 * 2 * 1. Note that 5! = 5 * 4!

So we have a ***recursive definition*** of factorial which is: x! = x * (x – 1)!

All recursive definitions (and recursive functions) must have a ***base case***. A base case is the one time that the definition (or function) does ***not*** use itself. Thus the base case will cause the recursion to stop. For the recursive factorial definition, the ***base case*** is this: 0! = 1 (the factorial of zero is equal to one).

A program using the recursive factorial function

```
#include <iostream>
using namespace std;

int factorial(int x);

int main() {
  int n, f;
  cout << "Enter a positive integer value ";
  cin >> n;
  f = factorial(n);
  cout << "The factorial of " << n << " is " << f << endl;
}

int factorial(int x) {
  int result;
  if (x == 0) {
    return 1;    // the base case
```

```
  }
  result = x * factorial(x - 1);
  return result;
}
```

Important points to note in this example of recursion

1. The function factorial calls itself – this is the classic definition of recursion
2. There are no loops in this program (or function). Recursion provides an alternative to loops.
3. Note that the recursive function has a base case (if x equals zero). The base case usually appears as an if-statement with a return inside it. It should appear before the recursive function call.
4. You do not have to use recursion to calculate factorial. As a homework challenge, write this program without using recursion. (Hint – you will need a loop). In fact, recursion wastes time by needing an excessive number of function calls. We don't use recursion to gain speed – we use recursion when the alternative algorithm is very complicated. You will learn this in later courses.
5. Factorials get very big, very quickly. The biggest value of n you can handle in this program is 16. Try it with 17 (or more) and see what happens. It is important for programmers to know the maximum integer (maxint) for the system they are using. Otherwise you can have a program that works perfectly but suddenly gives weird random results if the numbers get too big.

A walkthrough of this program

Let us assume that we want to calculate the factorial of 3.

main: calls the factorial function with n = 3

call 1: The function is called with x = 3
The function calculates: result = 3 * factorial(2) **
The function needs to "remember" where it is and then…

call 2: The function is called with x = 2
The function calculates: result = 2 * factorial(1) **
The function needs to "remember" where it is and then…

call 3: The function is called with x = 1
The function calculates: result = 1 * factorial(0) **
The function needs to "remember" where it is and then…

call 4: The function is called with x = 0
The function returns the value of 1 (the base case)

so…

call 3 can now calculate a result of 1 * 1 and return 1 to call 2

call 2 can now calculate a result of 2 * 1 and return 2 to call 1

call 1 can now calculate a result of 3 * 2 and return 6 to main

** denotes several "chunks of information" that the computer needs to "remember" as it goes through the recursive process. A computer "remembers" information by saving data in variables in memory. In order to do recursion the computer sets aside a section of the memory (called the ***stack*** – we will look at stacks later on in the course) in which it saves the local variables, parameters, etc for each recursive function call that it makes. If there are too many recursive function calls, it is possible that a ***stack overflow error*** may occur.

Two Approaches to Programming

We noted that this example program did not use loops. In general there are two approaches to programming – we can use recursion or we can use loops. Often both these approaches are available, e.g. we could write a different program (without recursion) that uses a for loop to calculate factorial. For each problem, we have to decide which approach will be the easiest and/or the most efficient to use. As programmers it is important to know that the two approaches are both available.

If an algorithm uses recursion then it is called a ***recursive*** algorithm (or program).

If an algorithm uses loops then it is called an ***iterative*** algorithm (or program).

Iteration refers to the loop-process, i.e. repeating the same instructions.

Another way of saying all this is that an iterative algorithm (or program) is one that does ***not*** use recursion. Most of the programs you have written up until now are iterative programs.

Linear Search and Binary Search

Searching is one of the most common activities carried out on computers. People are always looking up student records, bank account records, medical records, etc, etc.

Let us assume we have an array of (3-digit) ID numbers, like this:

array:

0	1	2	3	4	5
846	109	528	761	391	220

and so on….

Note that this array could be an array of objects (data records) but right now we are only interested in the ID numbers. As usual, each ID number is in an element of the array and has a ***position*** (or index). For example, the ID number 761 can be found at index 3 of the array.

Now assume that we want to ***search through the array*** for a particular ID number. There are at least two different searching algorithms available to us, as discussed below:

1) The Linear Search Algorithm

Start at the first element of the array (the element with an index of zero). Move through the array looking at each element. Check if each element matches the number you are looking for. "Linear" means "in a line" – it is called Linear Search because we move in a line through the data.

The Linear Search program

```
#include <iostream>
using namespace std;
```

```
int linearsearch(int first, int last, int target);
int loadrandomdata();

int id[500];
int total;     // number of ID numbers in the array
int target;    // the ID number that we are looking for
int indx;     // index of the ID number we are looking for

int main() {
  total = loadrandomdata();
  cout << "Enter an id to search for ";
  cin >> target;
  indx = linearsearch(0, total - 1, target);
  if (indx == -1) {
    cout << "The id " << target << " is not in the array.\n";
  } else {
    cout << "The id " << target << " is at index " << indx << endl;
  }
}

int linearsearch(int first, int last, int target) {
// first = first index, last = last index
  int i;
  indx = -1;
  for (i = first; i <= last; i++) {
    if (target == id[i]) {
      indx = i;
    }
  }
  return indx;
}

int loadrandomdata() {
  int num;
  // load the data randomly - feel free to use more numbers
  num = 6;
  id[0] = 846;
  id[1] = 109;
  id[2] = 528;
  id[3] = 761;
  id[4] = 391;
  id[5] = 220;
  return num;
}
```

Important points about the Linear Search program:

- we could search through a much bigger array (but you get the picture).
- loadrandomdata is a useful function – easy to modify if you want to load data in a different way. The load function is separate from main to help keep the focus on main.
- the linear search code runs ("in a line") from the first index to the last index.
- `indx` ends up with -1 if we cannot find the ID in the array.
- the major advantage of linear search – it works on random data and no sorting is needed.

- we often work with the ***indexes*** of an array. The function returns an ***index***. The parameters `first` and `last` are ***indexes***. They show us which part of the array we are looking at while using the function.

2) The Binary Search Algorithm

For a much larger array, linear search can take a long time (do we care?). A much faster alternative is called the Binary Search Algorithm. First of all, the data must be sorted. Once it is sorted we look in the middle of the array. If the ID number in the middle is less than the ID number we are looking for, we know that we only have to search through the second half of the array, and so on…

It is easy to see that Binary Search will always be faster than Linear Search. However Linear Search has one big advantage: the data does not have to be sorted. Sometimes, if data is being constantly inserted (or deleted) in the array, it is better to use Linear Search rather than Binary Search.

The important thing as a programmer is to know all the alternative options that are available to you.

The Binary Search program

```
#include <iostream>
using namespace std;

int binarysearch(int first, int last, int target);
int loadsorteddata();

int id[500];
int total;     // number of ID numbers in the array
int target;    // the ID number that we are looking for
int indx;     // index of the ID number we are looking for

int main() {
  total = loadsorteddata();
  cout << "Enter an id to search for ";
  cin >> target;
  indx = binarysearch(0, total - 1, target);
  if (indx == -1) {
    cout << "The id " << target << " is not in the array.\n";
  } else {
    cout << "The id " << target << " is at index " << indx << endl;
  }
}

int binarysearch(int first, int last, int target) {
// first = first index, last = last index
  int middle, result;
  if (first > last) {
    return -1;   // the base case
  }
  result = -1;
  middle = (first + last) / 2;
  if (target == id[middle]) {
    result = middle;
  } else if (target > id[middle]) {
    result = binarysearch(middle + 1, last, target);
```

```
  } else if (target < id[middle]) {
    result = binarysearch(first, middle - 1, target);
  }
  return result;
}

int loadsorteddata() {
  int num;
  // load data in SORTED order - or load randomly and sort it
  num = 6;
  id[0] = 109;
  id[1] = 220;
  id[2] = 391;
  id[3] = 528;
  id[4] = 761;
  id[5] = 846;
  return num;
}
```

Important points about the Binary Search program:

- the main program of the Binary Search program is almost exactly the same as the main program for Linear Search. That is the point – we want to compare the two searching algorithms and we use the same main program.
- the data has to be sorted for Binary Search to work. You can either load it in sorted order (as in the program) or you can load random data and then sort it.
- the binarysearch function is recursive – it calls itself. Note that it has no loops.
- like all good recursive functions, the binarysearch functions has a *base case*
- once again, we are working with the ***indexes*** of the array. The parameters `first` and `last` are ***indexes***. The local variable middle is also an index – it is the average of the first and last indexes. This allows us to look at the ID number that is in the middle of the section of interest.

Challenge – write a version of Binary Search without recursion

Binary Search naturally lends itself to recursion. However it is possible to write a binary search function without recursion (i.e. an ***iterative*** Binary Search function). Of course, it is sitting out there on the internet but we don't want to just take the easy way out...

Comparing Algorithms

Comparing algorithms is something that Computer Scientists spend a lot of time doing. Very often, there is more than one approach to a problem and people develop different algorithms to solve the problem. In this case we want to search through a list of ID numbers and we have linear search versus binary search.

Advantages of Linear Search: easy to program, data does not have to be sorted.
Advantages of Binary Search: complicated recursive program, data has to be sorted, faster than Linear

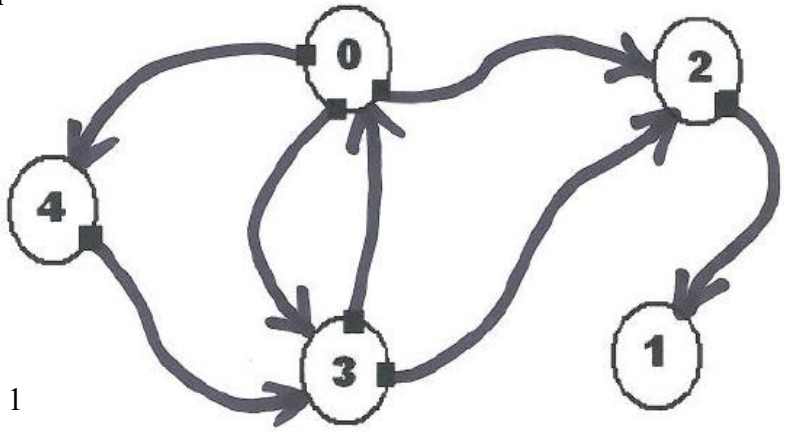
Which algorithm would you choose? Experienced programmers will always give high marks to "easy to program" because there is less chance of making a mistake. But speed may be important...

In real life there are often many different algorithms available and the differences may be difficult to see. For example, write a program to model climate change. Which is the best algorithm to use...?

Backtracking

Backtracking is the process of attempting to find a solution, realising that the current attempt will fail and then going back to a previous situation and trying a different solution. It is used in many games and Artificial Intelligence problems.

For example, assume a system of caves where each cave has a number of one-way doors with each door leading to another cave.



So cave 3 has 2 doors with one door leading to cave 0 and another door leading to cave 2.

We can now ask questions like:

Is there a path from cave 4 to cave 1?
Answer: yes, you can go 4 → 3 → 2 → 1
or you can go 4 → 3 → 0 → 2 → 1

Is there a path from cave 2 to cave 4?
Answer: no

Now imagine a much bigger system of thousands of caves with many links between them. It rapidly becomes very complicated to find a path between two caves. We need to use backtracking – start following a path but if we find that this path does not go anywhere, then we have to go back to a previous cave and try a different path.

The Caves Data Structure

Set up a `cave_class` that stores the following data for each cave:

```
int numdoors;   (the number of doors out of the cave)
int door[25];   (each door holds the number of the cave through the door)
```

Note that we assume a maximum of 25 doors from any given cave. This is an arbitrary number and you could change it if you want to.

As an example (using the diagram above), the data for cave 3 would be:

```
numdoors = 2   (there are 2 doors out of cave 3)
door[0] = 0;   (door 0 in cave 3 leads to cave 0)
door[1] = 2;   (door 1 in cave 3 leads to cave 2)
```

Note that the order of the doors is not important. We could equally have said that door 0 in cave 3 leads to door 2 and that door 1 leads to cave 0.

The Caves Program – Version 1

```
#include <iostream>
using namespace std;
```

```
void loadalldata();
bool findpath(int here, int last);

class cave_class {
private:
  int numdoors;  // number of doors out of the cave
  int door[25];  // hold index number of cave behind the door
public:
  void loaddata(int n, int a, int b, int c, int d, int e);
  int getnumdoors() { return numdoors; }
  int getdoor(int i) { return door[i]; }
};

int cave_total;        // total number of caves
cave_class cave[500];  // assume a maximum of 500 caves

int main() {
  int first, last;
  bool pathexists;
  loadalldata();
  cout << "Enter the number of the first cave: ";
  cin >> first;
  cout << "Enter the number of the last cave: ";
  cin >> last;
  cout << endl;
  pathexists = findpath(first, last);
  cout << last << endl;   // DEBUG cout
  if (pathexists == true) {
    cout << "A path was found from " << first;
    cout << " to " << last << "\n\n";
  } else {
    cout << "There is no path from " << first;
    cout << " to " << last << "\n\n";
  }
}

bool findpath(int here, int last) {
  int cavenum, num, i;
  bool success;
  if (here == last) {
    return true;      // the base case
  }
  num = cave[here].getnumdoors();
  cout << here << " -> ";  // DEBUG cout
  for (i = 0; i < num; i++) {
    cavenum = cave[here].getdoor(i);
    success = findpath(cavenum, last);
    if (success == true) {
      return true;
    }
  }
  return false;
}
```

```
void loadalldata() {
// load the data for all caves - change as needed
// this data is for the example in the notes
// -1 is used to indicate no door
  cave[0].loaddata(3, 4, 3, 2, -1, -1);
  cave[1].loaddata(0, -1, -1, -1, -1, -1);
  cave[2].loaddata(1, 1, -1, -1, -1, -1);
  cave[3].loaddata(2, 0, 2, -1, -1, -1);
  cave[4].loaddata(1, 3, -1, -1, -1, -1);
  cave_total = 5;
}

//--------------- methods for the cave_class ------------
void cave_class::loaddata(int n, int a, int b, int c, int d, int e)
{
// WARNING - does not work for more than 5 doors
  numdoors = n;
  door[0] = a;
  door[1] = b;
  door[2] = c;
  door[3] = d;
  door[4] = e;
}
```

But we have a problem...

The findpath function can go into an infinite loop. For example, find a path from cave 0 to cave 1. The function may try to follow this path: 0 → 4 → 3 → 0 → 4 → 3 → 0 → 4 → 3 → … This can happen even though another simple path exists. We need to be able to "mark" any cave that has been checked and then not check that particular cave again. (It is important to carry a piece of chalk with you when you wander through caves.)

To achieve this, we add the following to the program:
place a new boolean data item called `marked` in each cave
include new methods to get and set the variable `marked`
at the start of the program, remove all marks from caves (set marked to false for every cave)
in the findpath function, check if a cave has been previously marked – if it has, ignore that cave
in the findpath function, set marked to true for a cave before checking the doors out of the cave

Here is the final version of the program that marks caves that have been visited. Can you find a path that works correctly in version 2 of the program but did not work correctly in version 1?

The Caves Program – Version 2 (the program now marks the caves it visits)

```
#include <iostream>
using namespace std;

void loadalldata();
void removeallmarks();
bool findpath(int here, int last);

class cave_class {
```

```
private:
  int numdoors;  // number of doors out of the cave
  int door[25];  // hold index number of cave behind the door
  bool marked;   // mark a cave when you visit it
public:
  void loaddata(int n, int a, int b, int c, int d, int e);
  int getnumdoors() { return numdoors; }
  int getdoor(int i) { return door[i]; }
  void setmark(bool b) { marked = b; }
  bool getmark() { return marked; }
};

int cave_total;        // total number of caves
cave_class cave[500];  // assume a maximum of 500 caves

int main() {
  int first, last;
  bool pathexists;
  loadalldata();
  removeallmarks();
  cout << "Enter the number of the first cave: ";
  cin >> first;
  cout << "Enter the number of the last cave: ";
  cin >> last;
  cout << endl;
  pathexists = findpath(first, last);
  cout << last << endl;   // DEBUG cout
  if (pathexists == true) {
    cout << "A path was found from " << first;
    cout << " to " << last << "\n\n";
  } else {
    cout << "There is no path from " << first;
    cout << " to " << last << "\n\n";
  }
}

bool findpath(int here, int last) {
  int cavenum, num, i;
  bool success, mark;
  if (here == last) {
    return true;      // the base case
  }
  num = cave[here].getnumdoors();
  mark = cave[here].getmark();
  if (mark == true) {
    return false;   // do not visit a marked cave
  }
  cout << here << " -> ";  // DEBUG cout
  cave[here].setmark(true);  // mark the cave
  for (i = 0; i < num; i++) {
    cavenum = cave[here].getdoor(i);
    success = findpath(cavenum, last);
    if (success == true) {
      return true;
    }
```

```
  }
  return false;
}

void loadalldata() {
// load the data for all caves - change as needed
// this data is for the example in the notes
// -1 is used to indicate no door
  cave[0].loaddata(3, 4, 3, 2, -1, -1);
  cave[1].loaddata(0, -1, -1, -1, -1, -1);
  cave[2].loaddata(1, 1, -1, -1, -1, -1);
  cave[3].loaddata(2, 0, 2, -1, -1, -1);
  cave[4].loaddata(1, 3, -1, -1, -1, -1);
  cave_total = 5;
}

void removeallmarks() {
// remove the marks from all the caves
  int i;
  for (i = 0; i < cave_total; i++) {
    cave[i].setmark(false);
  }
}

//--------------- methods for the cave_class ------------
void cave_class::loaddata(int n, int a, int b, int c, int d, int e)
{
// WARNING - does not work for more than 5 doors
  numdoors = n;
  door[0] = a;
  door[1] = b;
  door[2] = c;
  door[3] = d;
  door[4] = e;
}
```

Backtracking is done by using recursion

We note that the findpath function calls itself – it is a recursive function. Backtracking is an example where recursion is the natural way to approach the problem. It is possible to write a non-recursive program to do backtracking but it would be extremely difficult and is not recommended!

Real-life Path and Backtracking Problems

We use the caves as a simple way of stating a complicated problem. In fact, computer programs all around us are tackling this sort of problem every day. Some examples are: the program in a GPS has to find a (shortest?) path between two buildings in a city; a milk delivery company wants the most efficient route between towns in New Zealand; we need to work out the cheapest way to lay fibre-optic cable (or sewerage pipes) between buildings on the campus; etc, etc. For more advanced algorithms to tackle these real-life problems, it is important for you to study 159.201.

We also use backtracking in scheduling problems. For example, in the university timetable, we might find class times for Maths, Physics and Programming. Then we may find that students will have a clash at a certain time. So we backtrack (perhaps remove Maths and Programming from the timetable) and try a different path (put Maths and Programming back into the timetable at a different time).